

Universidad CENFOTEC



Escuela de Software

Código del curso: SOFT-04.

Nombre del curso: Programación Orientada a Objetos.

Sección: SCV0.

Periodo: C1-2026.

Docente facilitador: Romario Salas Cerdas.

CONSIGNA DEL SEGUNDO AVANCE DE LA APLICACIÓN DE SOFTWARE

1. Datos generales de la actividad

Tipo de actividad:	Avance de proyecto.		
Fecha de entrega:	10 de abril del 2026, 11:59pm.	Valor porcentual:	15%.
Formato de entrega:	Archivo .pdf , commit de código en repositorio de GitHub.	Puntaje total:	90.
Individual: No.	Grupal: Sí.		

2. Instrucciones generales

1. Lea cuidadosamente las instrucciones de la actividad. En caso de tener alguna duda, puede consultar con su docente.
2. Esta actividad se desarrolla de manera grupal. Cualquier intento de plagio será sancionado de acuerdo con el reglamento académico vigente.
3. Al completar la actividad, un representante del grupo debe realizar el commit del código a GitHub y subir un archivo **.pdf** a la plataforma Moodle en el tiempo y espacio indicado por el docente. El archivo **.pdf** debe contener el diagrama de clases UML. El repositorio debe incluir todo el código **.java**.

3. Objetivos o competencias del curso que se evaluarán en la actividad de aprendizaje

Objetivo general o competencia del curso	Diseña una aplicación de software con el paradigma orientado a objetos, usando relaciones entre objetos y clases, modelados de aplicaciones, bases de datos e interfaces.
Objetivos específicos que	• Crea un repositorio de la aplicación en una

se evalúan	plataforma de control de versiones en línea para tener una versión inicial del software. • Confecciona el diagrama de lenguaje unificado de modelado (UML), incluyendo las clases, métodos y atributos de los objetos. • Elabora una estructura básica del código en un lenguaje de alto nivel, que contiene los paquetes de la capa lógica y el diseño de interfaz. • Elabora el diagrama de lenguaje unificado final (UML), incluyendo asociación, agregación, composición y herencia. • Desarrolla un repositorio de la aplicación en la plataforma para la lógica de las clases, incluyendo los paquetes, lógica de negocio y entidades. • Verifica la funcionalidad de la aplicación, utilizando como mínimo los dos casos de prueba. • Realiza una explicación de la aplicación y contesta de forma correcta las preguntas propuestas por el docente. • Incorpora la retroalimentación que el docente brindó a los diferentes avances.
--------------------------	---

4. Descripción de la actividad

En el segundo avance de la aplicación de software, los grupos deben desarrollar lo presentado en el primer avance aplicando el resto de los contenidos del paradigma de programación orientada a objetos vistos desde entonces, hasta implementar la funcionalidad completa de la aplicación tal cual se describe en la consigna del primer avance (i.e., debe demostrarse una aplicación plena de los conceptos de abstracción, encapsulamiento, polimorfismo, relaciones entre objetos y entre clases y modularidad). No obstante, la aplicación desarrollada para este entregable todavía no debe incluir la implementación de los patrones de diseño MVC y DAO, ni tampoco la persistencia de los datos en la base de datos MySQL. El diagrama de clases UML presentado en el primer avance debe ser modificado y actualizado según corresponda para reflejar los nuevos mecanismos de POO aplicados en el nuevo avance (e.g., clases abstractas, interfaces, modularidad, &c.).

Entregables:

1. Se debe incluir el código fuente completo de todas las clases desarrolladas.
2. Deben contar con un repositorio en GitHub para el proyecto de la interfaz gráfica y para el proyecto de la lógica de negocio.

3. Se debe incluir el diagrama UML de la capa lógica que refleje correctamente el diseño orientado a objetos implementado.
4. Se debe incluir **Javadoc** para todas las librerías generadas por el estudiante.
5. El proyecto debe implementar la arquitectura definida en el curso, respetando la separación entre capas.
6. Código legible y documentado (atributos privados, getters y setters, uso de **toString()** cuando aplique).

5. Rúbrica

Esta actividad de aprendizaje será evaluada mediante la siguiente rúbrica:

Criterio de evaluación	Insuficiente (1 punto)	Aceptable (2 puntos)	Bueno (3.5 puntos)	Excelente (5 puntos)
Modelado orientado a objetos y diagrama UML	El diagrama UML es inexistente o presenta errores graves de notación y diseño.	El diagrama incluye los elementos principales, pero presenta inconsistencias en relaciones o definición de métodos.	El diagrama representa correctamente las clases y la mayoría de las relaciones con notación adecuada.	El diagrama UML es claro, completo y correcto, representando fielmente el diseño orientado a objetos del sistema.
Encapsulamiento de atributos	Los atributos no están encapsulados o se manejan de forma incorrecta.	Se utiliza encapsulamiento de forma parcial o inconsistente.	Los atributos están correctamente encapsulados, con leves detalles por mejorar.	Todos los atributos están correctamente encapsulados, con uso consistente de modificadores de acceso y getters/setters.
Constructor por defecto	No se implementa el constructor por defecto o su implementación es incorrecta.	El constructor por defecto existe, pero presenta errores o uso limitado.	El constructor por defecto está correctamente implementado y funcional.	El constructor por defecto está correctamente implementado y se utiliza de forma coherente dentro del diseño.
Constructor	No se implementa	El constructor	El constructor	El constructor

general (con parámetros)	el constructor general o no asigna correctamente los atributos.	general existe, pero no asigna correctamente todos los atributos.	general asigna correctamente la mayoría de los atributos.	general asigna correctamente todos los atributos y refleja un diseño claro y consistente.
Implementación del método toString()	No se implementa el método toString() o no cumple su propósito.	El método toString() existe, pero muestra información incompleta o poco clara.	El método toString() muestra información relevante y bien estructurada.	El método toString() está correctamente implementado y facilita la visualización clara del estado del objeto.
Implementación de relaciones entre clases	Las relaciones no están implementadas o no coinciden con el modelo definido.	Las relaciones existen, pero presentan inconsistencias con el diseño UML.	Las relaciones están correctamente implementadas en su mayoría.	Todas las relaciones están correctamente implementadas y reflejan fielmente el diseño UML.
Arquitectura por capas	No se respeta la arquitectura definida y se mezclan responsabilidades entre capas.	Se intenta aplicar la arquitectura, pero existen errores en la separación de responsabilidades.	La arquitectura por capas está bien aplicada, con pequeños detalles por mejorar.	La arquitectura por capas se implementa correctamente, con separación clara y estricta entre capas.
Instanciación de objetos de negocio en la interfaz gráfica (UI)	La interfaz gráfica instancia directamente objetos de negocio de forma reiterada y sin respetar la arquitectura definida.	La interfaz gráfica instancia algunos objetos de negocio, aunque existe un intento parcial de delegar responsabilidades.	La interfaz gráfica instancia mínimamente objetos de negocio, existiendo pocos casos incorrectos.	La interfaz gráfica no instancia objetos de negocio; toda la creación y gestión de objetos se realiza exclusivamente en la capa lógica, respetando estrictamente la arquitectura definida.

Estructura de paquetes	La estructura de paquetes es incorrecta o no refleja la arquitectura definida.	La estructura de paquetes existe, pero presenta inconsistencias o desorden.	La estructura de paquetes es clara y en su mayoría coherente con la arquitectura.	La estructura de paquetes es clara, ordenada y refleja correctamente la arquitectura del sistema.
Registro de usuarios (menú)	La opción no funciona o no permite registrar usuarios correctamente.	Permite registrar usuarios, pero con errores de validación o flujo.	Permite registrar usuarios correctamente en la mayoría de los casos.	Permite registrar usuarios correctamente, aplicando validaciones y reglas de forma consistente.
Listado de usuarios (menú)	No se listan los usuarios o la información es incorrecta.	Se listan los usuarios, pero con información incompleta o desordenada.	Se listan correctamente los usuarios con información relevante.	El listado de usuarios es claro, completo y bien presentado.
Creación de subastas (menú)	No permite crear subastas o incumple reglas básicas.	Permite crear subastas, pero no valida correctamente todas las reglas.	Permite crear subastas aplicando la mayoría de las reglas del sistema.	Permite crear subastas aplicando correctamente todas las reglas definidas.
Listado de subastas (menú)	No se listan las subastas o la información es incorrecta.	Se listan las subastas, pero con información incompleta o poco clara.	Se listan correctamente las subastas con información relevante.	El listado de subastas es claro, completo y refleja correctamente el estado del sistema.
Creación de ofertas (menú)	No permite crear ofertas o incumple reglas básicas del sistema.	Permite crear ofertas, pero no valida correctamente todas las restricciones.	Permite crear ofertas aplicando la mayoría de las reglas del sistema.	Permite crear ofertas aplicando correctamente todas las reglas definidas.
Listado de ofertas (menú)	No se listan las ofertas o la información es	Se listan las ofertas, pero con información	Se listan correctamente las ofertas con	El listado de ofertas es claro, completo y

	incorrecta.	incompleta o desordenada.	información relevante.	consistente con el estado del sistema.
Persistencia en memoria con `ArrayList`	No se implementa correctamente la persistencia en memoria.	Se utiliza persistencia en memoria, pero con uso limitado o incorrecto de las estructuras de datos.	Se implementa correctamente la persistencia en memoria utilizando ArrayList , con manejo adecuado de los datos.	La persistencia en memoria está correctamente implementada, con uso claro, ordenado y consistente de ArrayList .
Calidad, legibilidad y documentación del código	Código desordenado, poco legible y sin documentación.	Código funcional, pero con problemas de legibilidad o documentación incompleta.	Código legible y bien documentado, con buenas prácticas básicas.	Código claro, ordenado y completamente documentado, incluyendo Javadoc y buenas prácticas.
Cumplimiento de entregables y reglas de la consigna	No cumple con los entregables mínimos o el proyecto no compila/ejecuta.	Cumple parcialmente con los entregables solicitados.	Cumple con la mayoría de los entregables, formatos, restricciones y reglas establecidas.	Cumple completamente con todos los entregables, formatos, restricciones y reglas establecidas en la consigna.